

TP_5

React Fundamentals – Functional Components, State, Props, Lists and User Input

Total duration: **90 minutes**

Independent student work: **~80 minutes**

Submission & short evaluation: **~10 minutes**

Learning Objectives

By the end of this TP session, students will be able to:

- Initialize a React project using Create React App
- Develop functional React components
- Use props for component communication
- Manage state with `useState`
- Render dynamic lists
- Handle controlled user input
- Implement filtering logic in a Single Page Application (SPA)

Theoretical Background

Students are expected to understand the following core concepts:

1. React as a Library

React is a JavaScript library for building user interfaces based on:

- Component-based architecture
- Declarative rendering
- Unidirectional data flow

2. Functional Components

```
function MyComponent() {  
  return <h1>Hello</h1>;  
}
```

or

```
const MyComponent = () => {
```

```
    return <h1>Hello</h1>;  
};
```

3. Props (Component Communication)

Props allow data transfer from parent to child components.

```
function Header({ title }) {  
    return <h1>{title}</h1>;  
}
```

4. State Management with useState

```
import { useState } from "react";  
  
const [value, setValue] = useState("");
```

State is used for:

- Storing dynamic data
- Triggering UI re-rendering

5. Event Handling

```
<input onChange={(e) => setValue(e.target.value)} />  
<button onClick={handleClick}>Click</button>
```

6. Rendering Lists

```
items.map(item => (  
    <li key={item.id}>{item.name}</li>  
))
```

Each list element must have a unique key.

7. Conditional Rendering

```
{items.length === 0 && <p>No tasks available</p>}
```

TP_Practical Assignment

Task Manager – Graduate Student Planner

Students must implement a small SPA for managing academic tasks.

Time Allocation

Stage	Activity	Time
	Project initialization	10 min
	Component structure setup	10 min
	Implement task creation	20 min
	Render dynamic task list	15 min
	Implement delete & toggle status	15 min
	Implement filtering	10 min
	Testing & refinement	10 min
Total		min

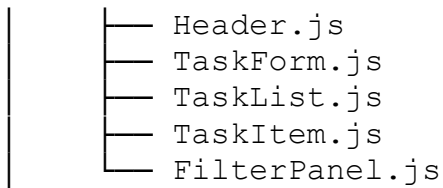
Application Requirements

The application must:

- Allow adding a new task via input field
- Display a dynamic list of tasks
- Allow:
 - Deleting a task
 - Marking a task as completed
- Display:
 - Total number of tasks
 - Completed tasks
 - Remaining tasks
- Implement filtering:
 - All
 - Active
 - Completed

Required Project Structure

```
src/  
├── App.js  
└── components/
```



Detailed Student Tasks

Stage 1 – Project Setup (10 min)

Create a new React project:

```
npx create-react-app task-manager  
cd task-manager  
npm start
```

Expected result:

- Running React application in browser

Stage 2 – Component Structure (10 min)

Create the following components:

- Header
- TaskForm
- TaskList
- TaskItem
- FilterPanel

Import and render them in `App.js`.

Expected result:

- Component layout visible in browser

Stage 3 – Task Creation (20 min)

Implement:

- Controlled input field

- `useState` for task text
- Add button
- Validation (no empty tasks)

Task object structure:

```
{  
  id: number,  
  text: string,  
  completed: boolean  
}
```

Expected result:

- New tasks appear in list
- Input clears after submission

Stage 4 – Rendering the List (15 min)

- Use `.map()` to render tasks
- Provide unique key
- Display message if list is empty

Expected result:

- Proper dynamic list rendering

Stage 5 – Delete & Toggle Status (15 min)

Add:

- Delete button
- Checkbox to toggle completion

Completed tasks must:

- Appear visually distinct (e.g., strikethrough)

Expected result:

- Correct status updates
- Correct deletion behavior

Stage 6 – Filtering (10 min)

Implement filtering logic using state:

- All
- Active
- Completed

Expected result:

- List updates dynamically without page reload

Stage 7 – Statistics & Final Testing (10 min)

Display:

- Total tasks
- Completed tasks
- Remaining tasks

Expected result:

- Correct real-time statistics
- No console errors

Assessment Criteria

Maximum score: 20 points

1. Architecture & Component Structure (4 points)

- Proper component separation (2 pts)
- Logical structure and data flow (2 pts)

2. State Management (4 points)

- Correct use of `useState` (2 pts)
- Immutable state updates (2 pts)

3. List Rendering (3 points)

- Correct use of `.map()` (2 pts)
- Unique `key` usage (1 pt)

4. User Input Handling (3 points)

- Controlled input implementation (2 pts)
- Validation logic (1 pt)

5. Filtering Logic (3 points)

- Correct filtering implementation (2 pts)
- Dynamic UI update (1 pt)

6. Code Quality & Clarity (3 points)

- Readable code
- Proper naming conventions
- No console errors

Bonus (up to +2 points, without exceeding 20)

Awarded for:

- Persistence using `localStorage`
- Clean UI styling
- Use of `useEffect`
- Additional feature (sorting, editing, etc.)

Expected Learning Outcomes

Students demonstrate:

- Understanding of SPA architecture
- Practical React development skills
- Component-based thinking
- Proper state handling
- Dynamic UI rendering
- Clean and maintainable code practices

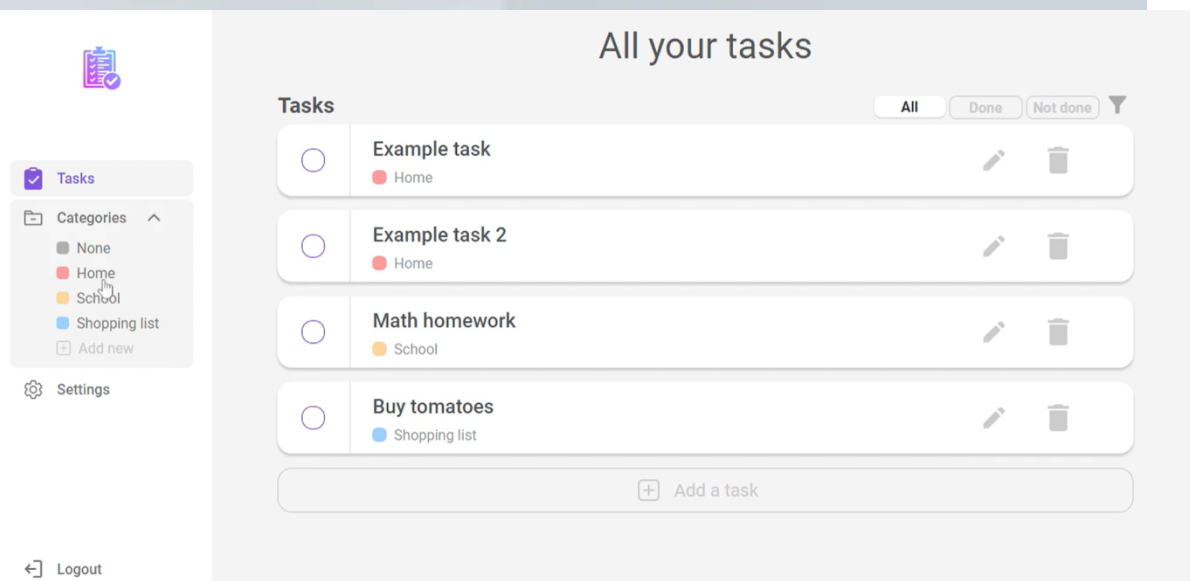
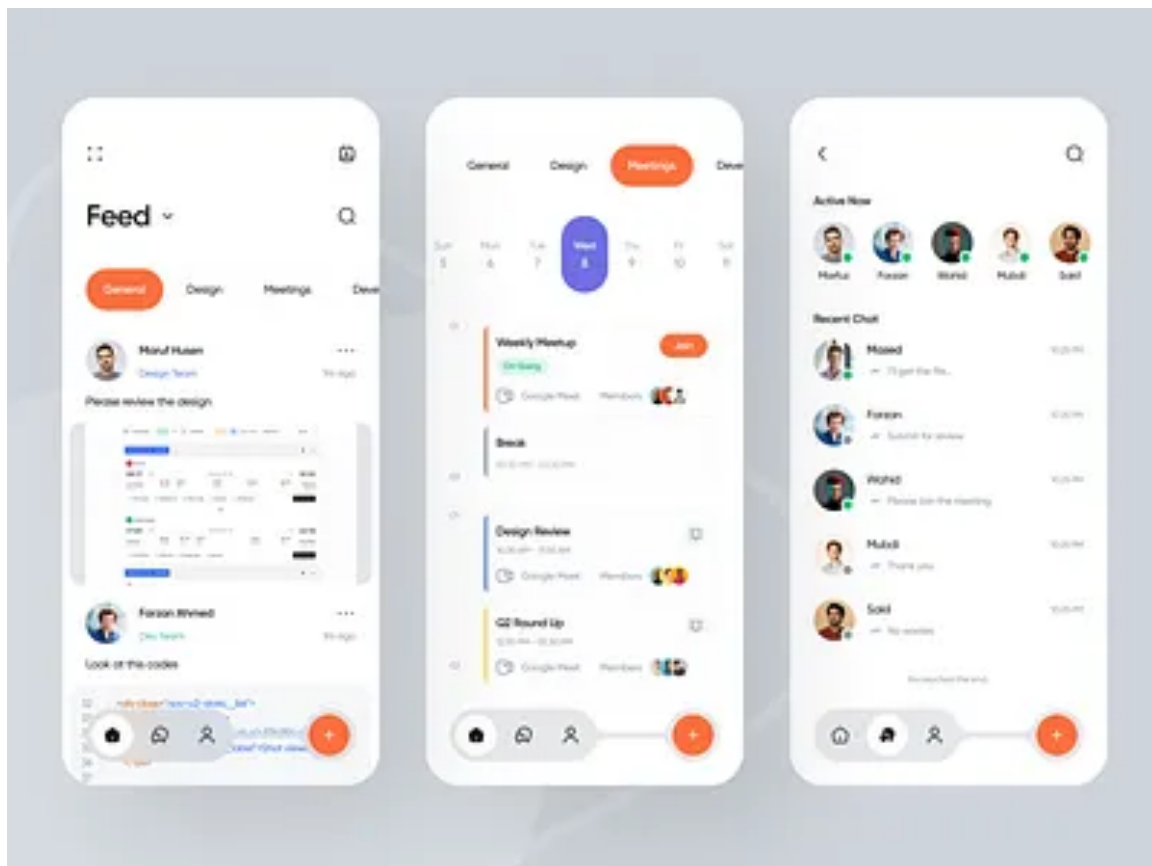
+++++

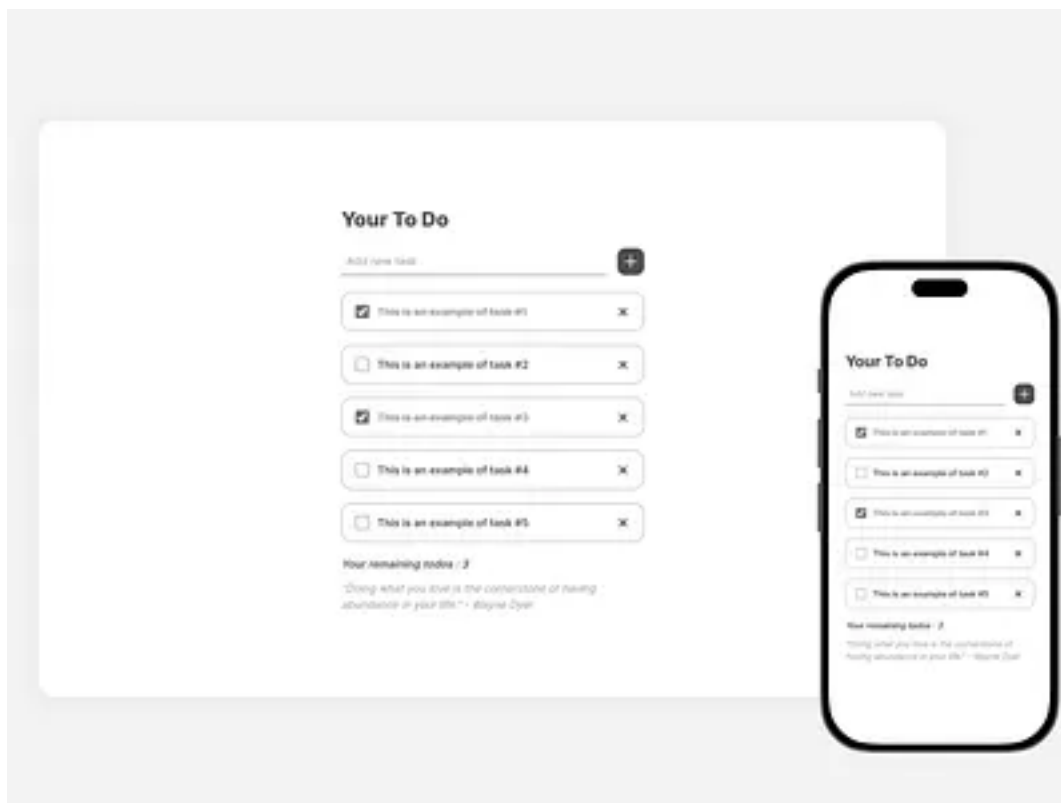
Expected Final Result

Task Manager – Graduate Student Planner (React SPA)

The final application must visually and functionally resemble the following structure.

1. Overall Layout





The page must contain:

1. **Application Title**
2. **Task Statistics**
3. **Task Input Form**
4. **Filter Buttons**
5. **Task List**

2. Functional Layout Structure

Header Section

Task Manager – Graduate Student Planner

Total: 5 | Completed: 2 | Remaining: 3

1. Must update dynamically
2. Must reflect real-time changes

Task Input Section

[Enter a new task here] (Add Task)

Requirements:

- Controlled input
- Button disabled OR validation prevents empty submission
- Input clears after submission

Filter Section

[All] [Active] [Completed]

Requirements:

- Only one filter active at a time
- Active filter visually highlighted
- No page reload
- Instant filtering

Task List Section

Each task must contain:

- | | |
|---|----------|
| ☐ Prepare research proposal | (Delete) |
| ☒ Read academic paper | (Delete) |

Where:

- Checkbox toggles completion
- Completed task:
 - text must be visually different (e.g., line-through)
- Delete button removes task immediately

3. Functional Behaviour (Strict Requirements)

Students must demonstrate:

✓ Adding Tasks

- Adds new task at top or bottom (consistent logic)
- No empty strings
- Each task has unique ID

✓ State Structure (Expected)

```
{  
  id: number,  
  text: string,
```

```
    completed: boolean
  }
```

✓ Immutable Updates

Example of correct update:

```
setTasks(prev =>
  prev.map(task =>
    task.id === id
      ? { ...task, completed: !task.completed }
      : task
  )
);
```

Students must NOT:

- Mutate arrays directly
- Use push/splice without copying state

✓ Empty State Handling

If no tasks exist:

No tasks available.

4. Minimum Component Structure (Required)

```
App
├── Header
├── TaskForm
├── FilterPanel
├── TaskList
│   └── TaskItem
```

Students must:

- Pass props properly
- Avoid placing all logic inside App without separation

5. What Will Be Considered Incorrect

No component separation
Direct state mutation
Missing key in list
No filtering
Page reload
Console errors
Using plain JavaScript without React state

6. Minimal Styling Expectations

This is a computer science Master's level lab —
Design is secondary, but UI must be:

- Structured
- Readable
- Not broken
- Without layout overlap
- Without raw default browser chaos

Minimal CSS is sufficient.

7. Final Demonstration Checklist (Students Must Show)

During submission, students must demonstrate:

1. Adding a task
2. Toggling completion
3. Deleting a task
4. Filtering tasks
5. Statistics updating
6. Code structure (components + useState)

Final Clarification for Students

The goal is not UI beauty.
The goal is correct:

- React architecture
- State management
- Dynamic rendering
- Component communication